

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Sessions 19–20

Security with Keycloak & OAuth2

Session 19 — Keycloak, OAuth2 Authorization Code & OpenID Connect

Session 20 — Client Credentials, Resource Server & Role-Based Authorization

1. What We Are Building

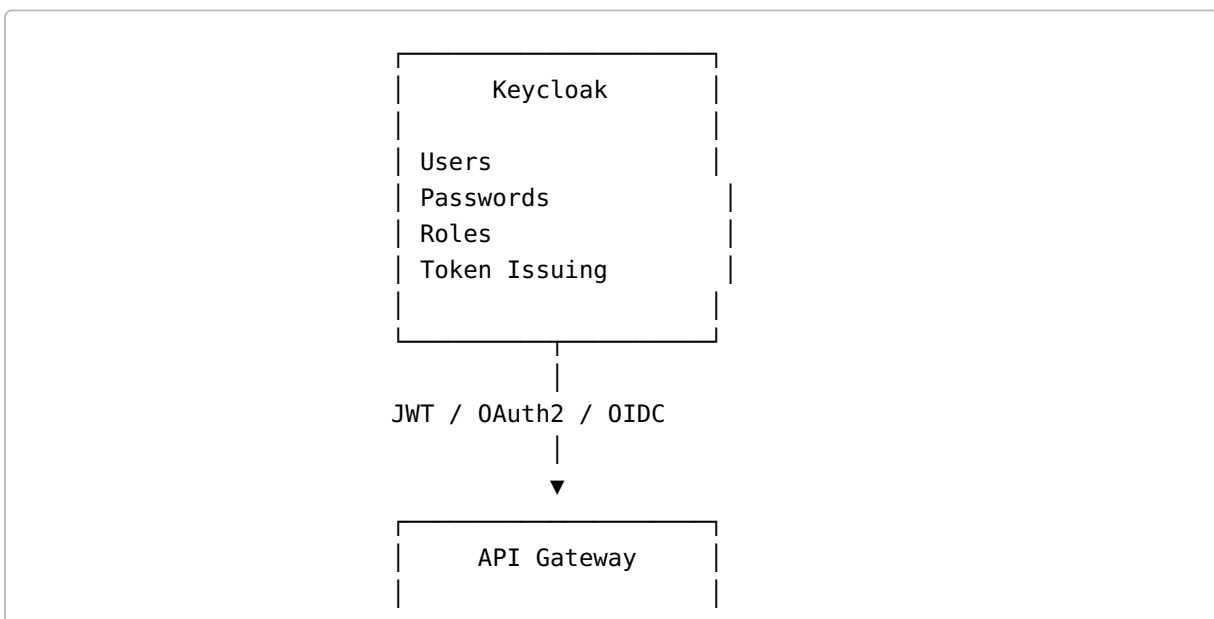
In the earlier security session, we created our own JWT authentication mechanism.

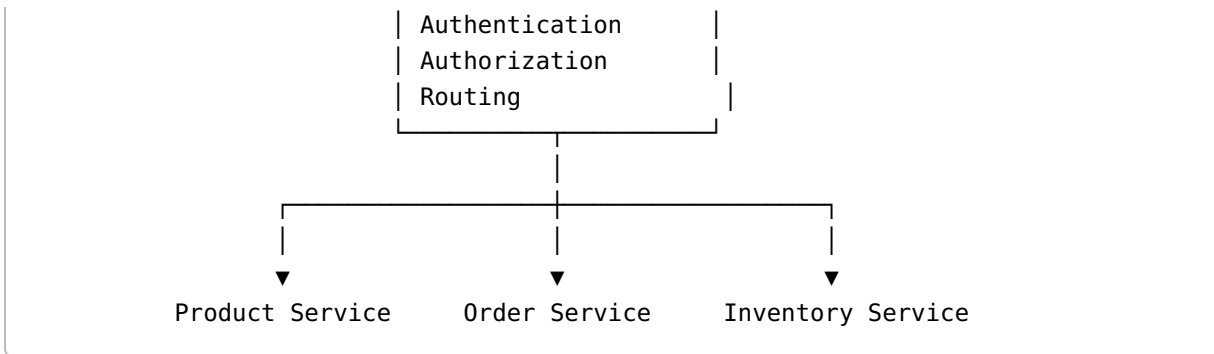
That was useful for learning how JWTs work, but it is not enough for a real enterprise application.

In these two sessions, we replace the hand-written security mechanism with a real Identity Provider:

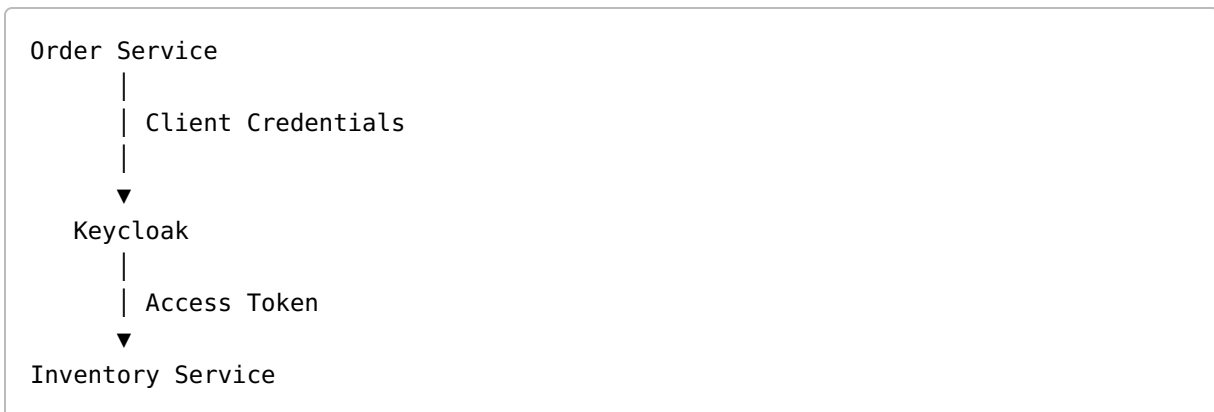
Keycloak

The final architecture will look conceptually like this:





We will also support service-to-service authentication:



PART A

SESSION 19

Security — Part 1

Keycloak & OAuth2 Authorization Code

2. Learning Objectives

By the end of Session 19, you should be able to:

Previous Security Limitations

You should be able to explain why our Session 3 security implementation was only a learning implementation.

You should understand:

- Why a hardcoded JWT secret is not appropriate for production.
- Why manually creating JWTs is not a real authentication system.
- Why our previous implementation had no user-management system.
- Why there was no password-reset mechanism.
- Why there was no login interface.
- Why revoking an individual JWT was difficult.
- Why an application needs a trusted Identity Provider.

OAuth2 and OpenID Connect

You should understand:

- OAuth2 roles.
- Authorization Server.
- Resource Server.
- OAuth2 Client.
- Resource Owner.
- Identity Provider.
- Authorization Code Flow.
- OpenID Connect.
- Access Token.
- ID Token.
- Refresh Token.
- Public Client.
- Confidential Client.
- PKCE.

Keycloak

You should be able to:

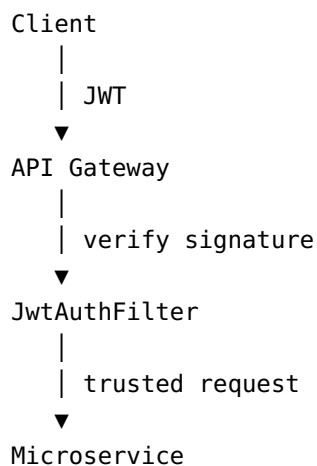
- Run Keycloak using Docker.
 - Create a Realm.
 - Create a Client.
 - Create a User.
 - Configure a client as confidential.
 - Understand the authorization endpoint.
 - Understand the token endpoint.
 - Obtain an authorization code.
 - Exchange an authorization code for tokens.
 - Inspect a JWT.
-

3. Why Are We Replacing Our Previous JWT Security?

3.1 What We Built Earlier

In Session 3 we created a JWT authentication mechanism.

Conceptually, our system looked like:



The Gateway knew the secret used to sign the JWT.

For example:

```
jwt:  
secret: microservices-pro-course-secret-key-2024-minimum-256-bits
```

This was useful for learning.

But imagine this system in production.

4. Problems with the Hand-Rolled Security

Problem 1 — Where do users come from?

We did not build:

```
Sign Up
Login
Logout
Password Change
Password Reset
Email Verification
Multi-Factor Authentication
```

We only created tokens for testing.

A real system needs a real identity system.

Problem 2 — Where are passwords stored?

Our microservices should not become responsible for:

```
User passwords
Password hashing
Password reset
Account locking
MFA
Login attempts
Identity verification
```

These are identity-management responsibilities.

Problem 3 — What happens if a token is stolen?

Suppose:

```
Access Token
expires in 60 minutes
```

An attacker steals it.

Our simple JWT validation checks:

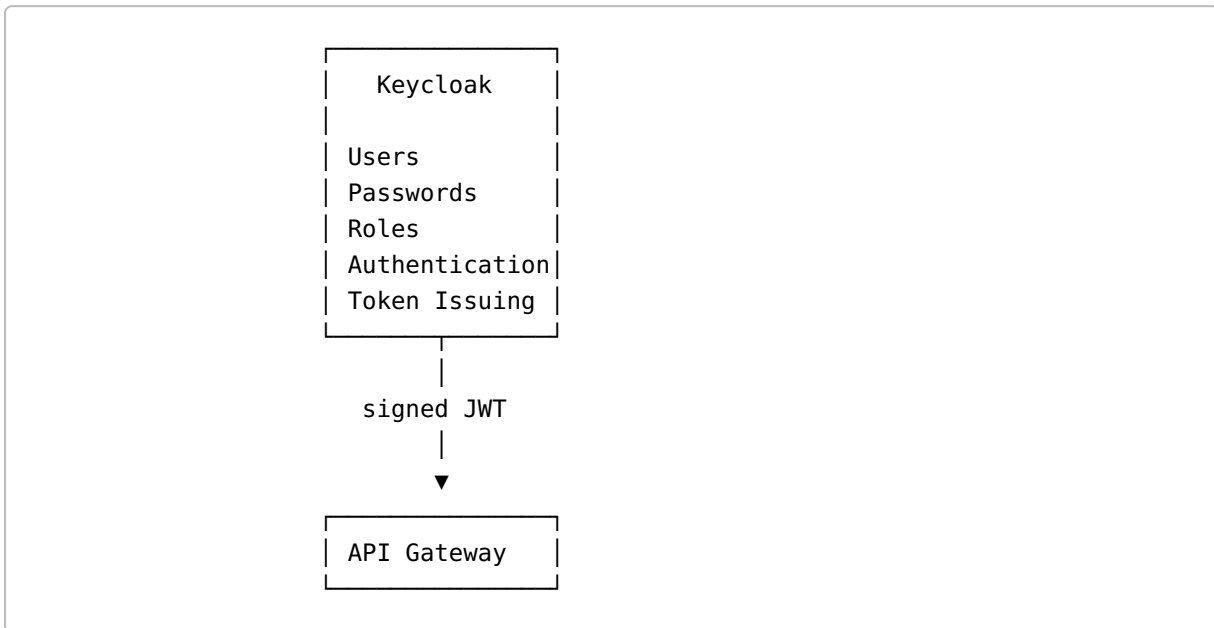
```
Is the signature valid?
Is the token expired?
```

If both are true, the token is accepted.

There is no central identity system managing the user's session or account.

5. The New Idea

Instead of every application creating and understanding user authentication, we introduce an Identity Provider.



Now our Gateway does not need to know:

- How the password was checked.
- Where the user was created.
- How password reset works.
- Whether MFA was used.

It only needs to trust tokens issued by Keycloak.

This is the important architectural shift:

Before:

Gateway trusts tokens because it knows a shared secret.

After:

Gateway trusts tokens because it trusts the issuer.

6. OAuth2 — The Big Picture

OAuth2 defines a framework for delegated authorization.

For our course, the most important concepts are:

Role	Meaning
Resource Owner	The user who owns the data
Client	Application requesting access
Authorization Server	System that issues tokens
Resource Server	API that accepts access tokens

In our example:

```
Resource Owner
  =
Customer

Authorization Server
  =
Keycloak

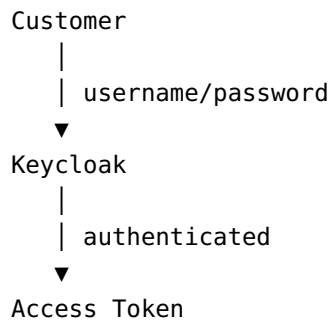
Client
  =
Application participating in the OAuth2 flow

Resource Server
  =
Protected API
```

7. Identity Provider

An Identity Provider, or IdP, is responsible for establishing identity.

For example:



The application does not need to implement all authentication logic itself.

Keycloak can manage:

- Users
- Passwords
- Roles
- Groups
- Authentication
- Login
- Token issuance
- Single Sign-On
- Multi-factor authentication

8. Keycloak

Keycloak is an open-source Identity and Access Management platform.

In this course we will use Keycloak as our:

```
graph TD; IP[Identity Provider] --- AS[Authorization Server]; AS --- OP[OpenID Connect Provider];
```

9. Realm

A Realm is an isolated security domain in Keycloak.

For our application:

```
Realm:
ecommerce-platform
```

Inside this realm we can have:

```
Users
Clients
Roles
Groups
Identity configuration
```

For example:

```
Keycloak
├── master
└── ecommerce-platform
    ├── Users
    │   ├── customer1
    │   └── admin1
    ├── Clients
    │   ├── gateway
    │   └── order-service
    └── Roles
        ├── USER
        └── ADMIN
```

Do not normally place application users in the `master` realm.

The `master` realm is primarily used for administering Keycloak.

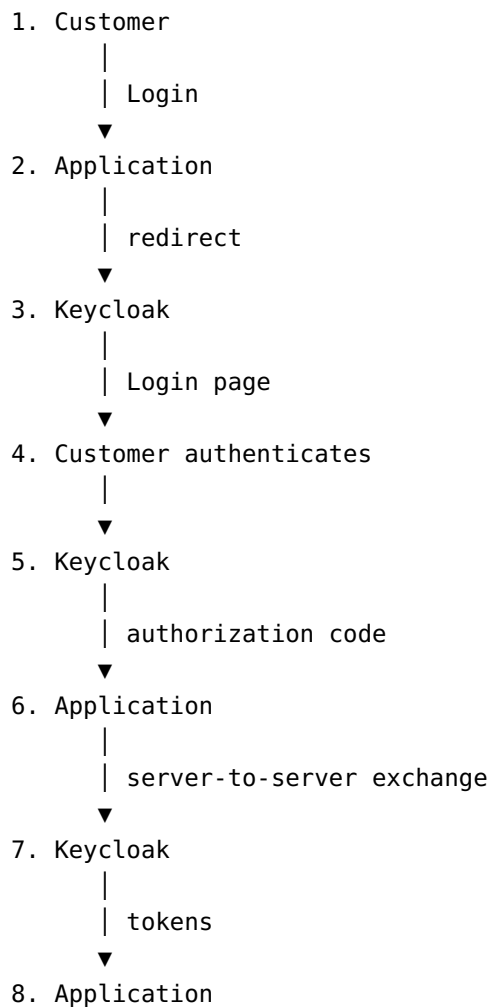
10. OAuth2 Authorization Code Flow

This is the most important flow in Session 19.

Suppose:

Customer wants to log in.

The high-level process is:



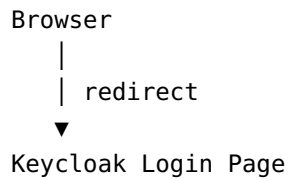
11. Why Does the Browser Participate?

A common beginner question is:

Why doesn't the application simply send the username and password to Keycloak?

Because the application should not collect the user's password when using this architecture.

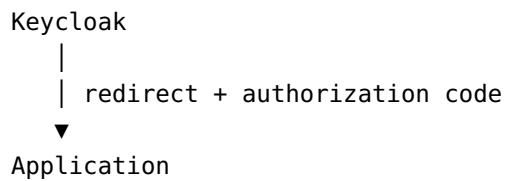
Instead:



The user enters the password directly into Keycloak.

The application never sees it.

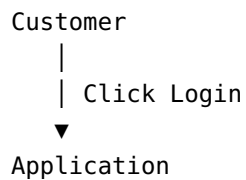
After successful authentication:



The authorization code is then exchanged for tokens.

12. Authorization Code Flow — Detailed

Step 1 — User starts login



Step 2 — Browser is redirected to Keycloak

Example:

GET

/realms/ecommerce-platform/protocol/openid-connect/auth

with parameters such as:

```
client_id=gateway
redirect_uri=http://localhost:8080/login/oauth2/code/keycloak
response_type=code
scope=openid
```

Step 3 — Keycloak displays login

The browser displays:

```
Username
Password
Login
```

The credentials are submitted to Keycloak.

The Gateway does not receive the password.

Step 4 — Keycloak authenticates the user

If successful:

```
customer1
  |
  ▼
authenticated
```

Step 5 — Keycloak returns an authorization code

The browser is redirected to:

```
http://localhost:8080/login/oauth2/code/keycloak?code=abc123
```

The code is:

- short-lived
- temporary
- intended to be exchanged for tokens

Step 6 — Application exchanges the code

The server sends a request to Keycloak:

```
POST /realms/ecommerce-platform/protocol/openid-connect/token
```

with:

```
grant_type=authorization_code
client_id=gateway
client_secret=...
code=abc123
redirect_uri=...
```

Step 7 — Keycloak returns tokens

Typically the response contains:

```
access_token
id_token
refresh_token
expires_in
```

13. Access Token vs ID Token

This distinction is extremely important.

Access Token

The Access Token is used to access protected APIs.

Conceptually:

```
Client
|
| Authorization: Bearer <access-token>
▼
Resource Server
```

The API should care primarily about the access token.

ID Token

The ID Token belongs to OpenID Connect.

It provides information about the authenticated user to the client.

For example:

```
User ID
Username
Authentication information
```

Do not teach students that an ID Token is simply another API access token.

14. OAuth2 vs OpenID Connect

Remember:

```
OAuth2
=
Authorization

OpenID Connect
=
Authentication/Identity on top of OAuth2
```

Therefore, our login scenario is better described as:

```
OAuth 2.0 Authorization Code Flow
+
OpenID Connect
```

15. Public vs Confidential Clients

OAuth2 clients can be deployed in different environments.

Confidential Client

A server-side application can keep a secret.

Example:

```
Backend
Gateway
Server-side application
```

It can have:

```
client_id
client_secret
```

Public Client

A browser application or mobile application cannot reliably keep a secret.

For example:

```
SPA
Mobile Application
Desktop Application
```

If we place:

```
client_secret=ABC123
```

inside JavaScript shipped to the browser, the user can inspect it.

Therefore, the secret is not actually secret.

16. PKCE

PKCE means:

```
Proof Key for Code Exchange
```

It protects the authorization-code exchange against code interception.

The simplified idea is:

```
Client
|
| code_challenge
|
▼
Authorization Server
|
| authorization code
|
▼
Client
|
| code + code_verifier
|
▼
Authorization Server
```

The authorization server verifies that the client that started the flow is the one completing it.

For modern applications:

```
Browser / Mobile
    ↓
Public Client + PKCE
```

is the normal approach.

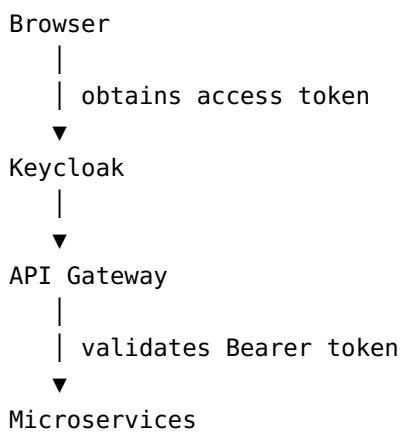
17. Our Course Architecture

For this course we will use a server-side Gateway/BFF-style example so that students can understand a confidential client.

However, remember:

The API Gateway does not have to be the OAuth2 Client in every architecture.

Another architecture is:



In Session 20, we will focus strongly on this Resource Server model.

18. Installing Keycloak

We will run Keycloak using Docker.

For course reproducibility, pin the Keycloak image version rather than using `latest`.

Example:

```
keycloak:
  image: quay.io/keycloak/keycloak:26.7.3
  command: start-dev
  environment:
    KC_BOOTSTRAP_ADMIN_USERNAME: admin
    KC_BOOTSTRAP_ADMIN_PASSWORD: admin
  ports:
    - "8180:8080"
```

```
networks:
  - platform-net
```

These credentials are for local development only.

Never use:

```
admin/admin
```

in production.

19. Start Keycloak

Run:

```
docker compose up -d keycloak
```

Check:

```
docker compose ps
```

Then open:

```
http://localhost:8180
```

Login:

```
Username: admin
Password: admin
```

20. Create the Realm

Open Keycloak Admin Console.

Create:

Realm Name:
ecommerce-platform

We will use:

ecommerce-platform

throughout the course.

21. Create the Gateway Client

Inside:

ecommerce-platform

create a client:

Client ID:
gateway

For our server-side demonstration:

Client authentication:
ON

This makes it a confidential client.

Enable the authorization-code/standard login flow appropriate to your Keycloak version.

Configure:

Valid redirect URI:

`http://localhost:8080/login/oauth2/code/keycloak`

Save the client.

Then open:

Credentials

and copy the client secret.

22. Create a Test User

Go to:

Users

Create:

Username:
customer1

Email:
customer1@example.com

Then configure a password.

For the classroom demonstration, a non-temporary password can be used.

For production:

- use strong passwords
 - do not share credentials
 - enable appropriate password policies
 - consider MFA
-

23. Perform the Authorization Flow Manually

For learning purposes, we will manually demonstrate the protocol.

Open:

<http://localhost:8180/realms/ecommerce-platform/protocol/openid-connect/auth>

with:

```
client_id=gateway
redirect_uri=http://localhost:8080/login/oauth2/code/keycloak
response_type=code
scope=openid
```

A properly URL-encoded version can be used in the browser.

24. Login

Login as:

```
customer1
```

After successful login, Keycloak redirects the browser to something similar to:

```
http://localhost:8080/login/oauth2/code/keycloak?code=...
```

The application is not necessarily running the callback in this manual exercise.

The purpose is to observe the protocol.

Copy the authorization code for the demonstration.

25. Exchange the Authorization Code

The authorization code must be exchanged for tokens.

Conceptually:

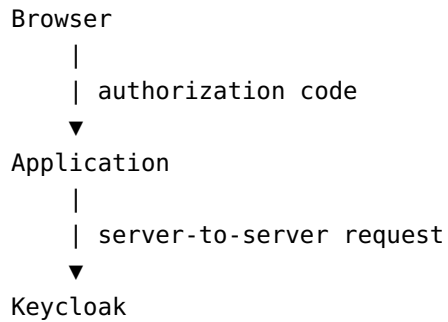
```
POST
/realms/ecommerce-platform/protocol/openid-connect/token
```

Parameters:

```
grant_type=authorization_code
client_id=gateway
```

```
client_secret=...
code=...
redirect_uri=...
```

The important point is:



The client secret must never be exposed to the browser.

26. Inspect the Access Token

You can inspect the JWT during the classroom demonstration.

A JWT has three parts:

```
HEADER
.
PAYLOAD
.
SIGNATURE
```

For example:

```
xxxxx.yyyyy.zzzzz
```

27. Important JWT Claims

A Keycloak token can contain claims such as:

```
iss
sub
exp
iat
realm_access
```

`iss`

Issuer.

It identifies who issued the token.

For example:

```
http://localhost:8180/realms/ecommerce-platform
```

`sub`

Subject.

Usually identifies the user or client represented by the token.

`exp`

Expiration time.

The token should not be accepted after this time.

`realm_access.roles`

Can contain Keycloak realm roles.

For example:

```
{
  "realm_access": {
    "roles": [
      "USER",
      "ADMIN"
    ]
  }
}
```

```
}  
}
```

28. The Major Security Improvement

Our previous system essentially said:

```
"I trust this JWT because I know the signing secret."
```

The new system says:

```
"I trust this JWT because it was issued by my trusted Identity Provider."
```

This is the key architectural lesson.

PART B

SESSION 20

Security — Part 2

Client Credentials & Resource Server

29. Session 20 Learning Objectives

By the end of this session, you should be able to:

- Explain service-to-service authentication.
- Explain Client Credentials Flow.
- Distinguish user tokens from service tokens.
- Configure a Keycloak service account.
- Obtain a service access token.
- Understand JWKS.
- Understand asymmetric JWT signing.
- Configure Spring Security OAuth2 Resource Server.

- Replace the hand-written JWT filter.
- Map Keycloak roles to Spring Security authorities.
- Protect routes using roles.
- Understand Authentication vs Authorization.

30. The Problem: What If There Is No User?

Consider this scenario.

A scheduled job runs every night:

Find products with low stock.

There is no customer.

Therefore:

No customer JWT exists.

The process is:

```
Scheduler
|
▼
Order Service
|
▼
Inventory Service
```

Who is making the request?

Not:

customer1

Instead:

order-service

The service itself needs an identity.

31. Client Credentials Flow

OAuth2 provides a flow for this:

Client Credentials Flow

The important difference is:

Authorization Code
=
Client acts on behalf of a user

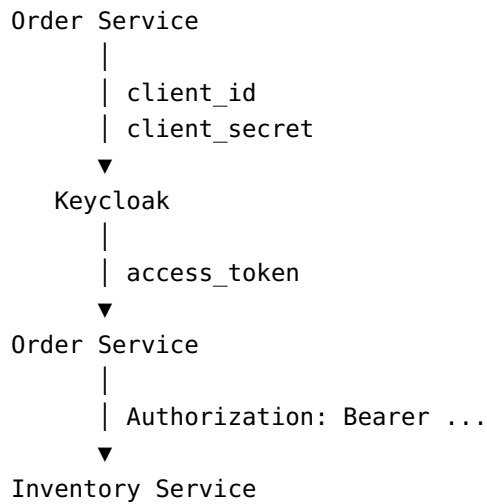
Client Credentials
=
Client acts as itself

32. Compare the Two Flows

	Authorization Code	Client Credentials
User involved?	Yes	No
Browser?	Usually yes	No
Redirect?	Yes	No
Authentication subject	User	Service/client
Typical use	Customer login	Service-to-service
Example	Customer → Gateway	Order → Inventory

33. Client Credentials Flow

The flow is simple:



No browser is required.

No user is involved.

34. Create the Order Service Client

In Keycloak:

Realm:
ecommerce-platform

Create:

Client ID:
order-service

Enable:

Client authentication

Enable the service-account/client-credentials capability appropriate to your Keycloak version.

You do not need the normal interactive browser login flow for this client.

Save the client.

Copy its client secret.

35. Request a Service Token

The token endpoint is:

```
POST
```

```
/realms/ecommerce-platform/protocol/openid-connect/token
```

Send:

```
grant_type=client_credentials  
client_id=order-service  
client_secret=...
```

There is:

```
NO username  
NO password  
NO redirect_uri  
NO authorization code
```

36. What Does the Token Represent?

This is extremely important.

For a customer:

```
sub = customer identity
```

For Client Credentials:

```
sub = service/client identity
```

Conceptually:

Customer Token:

"I am customer1."

Service Token:

"I am order-service."

37. JWKS

Now we reach the most important technical improvement in Session 20.

Our old JWT implementation used a shared secret.

For example:

```
Secret
|
├── Gateway
├── Order Service
└── Inventory Service
```

Every component that verifies tokens must know the secret.

This creates a security problem.

38. Asymmetric Signing

Keycloak can use asymmetric cryptography.

Conceptually:

```
Keycloak
|
└─ private key
   ▼
Signs JWT
```

The corresponding public key can be distributed.

```
Public Key
|
├── Gateway
├── Order Service
└── Inventory Service
```

The important rule is:

```
Private key:
ONLY Keycloak

Public key:
Can be distributed to token validators
```

39. JWKS

JWKS means:

```
JSON Web Key Set
```

It is a set of public keys published by the authorization server.

Keycloak exposes the keys through its OpenID Connect metadata.

For our realm, the JWKS endpoint is conceptually:

```
http://localhost:8180/realms/ecommerce-platform/protocol/openid-connect/certs
```

Spring Security can discover this information automatically when configured with the issuer URI.

40. Why Is This Better?

The old architecture:

```
Gateway
  ↓
Shared Secret
```

The new architecture:

```
Keycloak
  |
  | private key
  ▼
JWT
  |
  | public key
  ▼
Gateway
  |
  | public key
  ▼
verify JWT
```

The Gateway never receives Keycloak's private signing key.

41. Resource Server

A Resource Server is an application that protects resources and accepts access tokens.

For example:

```
Product Service
Order Service
Inventory Service
API Gateway
```

can act as Resource Servers.

Conceptually:

```
Client
  |
  | Bearer Token
  ▼
Resource Server
  |
```

```
| validate  
▼  
Allow / Reject
```

42. Spring Security Resource Server

Instead of writing:

```
JwtAuthFilter  
JwtUtil  
manual signature validation
```

we let Spring Security perform JWT validation.

Add:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>  
</dependency>
```

The Resource Server support uses JWT/Jose functionality to decode and verify JWTs. Spring Boot can configure the decoder from the authorization server's issuer URI.

43. Remove the Old JJWT Dependencies

Delete the hand-written JJWT dependencies used only by Session 3.

For example:

```
io.jsonwebtoken:jjwt-api  
io.jsonwebtoken:jjwt-impl  
io.jsonwebtoken:jjwt-jackson
```

If no other application code uses them, they should be removed.

44. Configure the Issuer

In:

```
application.yml
```

configure:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8180/realms/ecommerce-platform
```

Spring Security can use this issuer information to discover the authorization-server metadata and public keys, and validate the token's issuer and timestamps.

45. Why Do We Configure `issuer-uri`?

Suppose an attacker creates their own JWT.

It might have:

```
valid signature
```

but it was not issued by our Keycloak.

We don't want to trust it.

The token should contain:

```
iss = our trusted Keycloak realm
```

Spring Security verifies the issuer.

Therefore:

```
Valid signature
+
Trusted issuer
+
Valid timestamps
=
acceptable token
```

46. Issuer vs Audience

Do not confuse:

```
iss
```

with:

```
aud
```

Issuer

Answers:

Who issued this token?

```
iss = Keycloak
```

Audience

Answers:

Who is this token intended for?

```
aud = my-api
```

In a production architecture, audience validation can provide an additional protection layer. Spring Security supports configuring audiences in addition to the issuer.

47. Gateway Security Configuration

Our Gateway uses WebFlux.

Therefore we use:

```
@EnableWebFluxSecurity
```

not:

```
@EnableWebSecurity
```

A basic configuration is:

```
@Configuration
@EnableWebFluxSecurity
public class SecurityConfig {

    @Bean
    public SecurityWebFilterChain securityFilterChain(
        ServerHttpSecurity http) {

        return http
            .authorizeExchange(exchanges -> exchanges
                .pathMatchers(
                    HttpMethod.GET,
                    "/api/products"
                ).permitAll()

                .pathMatchers(
                    "/api/admin/**"
                ).hasRole("ADMIN")

                .anyExchange()
                .authenticated()
            )

            .oauth2ResourceServer(oauth2 ->
                oauth2.jwt(jwt ->
                    jwt.jwtAuthenticationConverter(
                        keycloakRoleConverter()
                    )
                )
            )
    }
}
```

```
        )  
    )  
    .build();  
}  
}
```

48. Understanding the Configuration

This:

```
.pathMatchers(  
    HttpMethod.GET,  
    "/api/products"  
) .permitAll()
```

means:

```
GET /api/products
```

does not require authentication.

This:

```
.pathMatchers("/api/admin/**")  
.hasRole("ADMIN")
```

means:

```
/api/admin/...
```

requires:

```
ROLE_ADMIN
```

This:

```
.anyExchange().authenticated()
```

means all remaining routes require authentication.

49. Authentication vs Authorization

This distinction must become second nature.

Authentication

Question:

Who are you?

Example:

```
customer1
```

Keycloak + JWT authentication solves this.

Authorization

Question:

Are you allowed to perform this operation?

Example:

```
customer1
```

might not be allowed to:

```
DELETE /api/products/10
```

while:

```
admin1
```

may be allowed.

Therefore:

Authentication

=

Who are you?

Authorization

=

What are you allowed to do?

50. Keycloak Roles

Suppose we have:

```
USER  
ADMIN
```

A Keycloak token might contain:

```
{  
  "realm_access": {  
    "roles": [  
      "USER",  
      "ADMIN"  
    ]  
  }  
}
```

Spring Security needs authorities in a form it understands.

For example:

```
ROLE_USER  
ROLE_ADMIN
```

51. Mapping Keycloak Roles

A converter can translate Keycloak roles.

Conceptually:

```
private Converter<Jwt, Mono<AbstractAuthenticationToken>>
keycloakRoleConverter() {

    JwtAuthenticationConverter converter =
        new JwtAuthenticationConverter();

    converter.setJwtGrantedAuthoritiesConverter(jwt -> {

        Map<String, Object> realmAccess =
            jwt.getClaim("realm_access");

        if (realmAccess == null) {
            return List.of();
        }

        Object rolesObject =
            realmAccess.get("roles");

        if (!(rolesObject instanceof Collection<?> roles)) {
            return List.of();
        }

        return roles.stream()
            .filter(Objects::nonNull)
            .map(Object::toString)
            .map(role ->
                new SimpleGrantedAuthority(
                    "ROLE_" + role.toUpperCase()
                )
            )
            .collect(Collectors.toList());
    });

    return new ReactiveJwtAuthenticationConverterAdapter(
        converter
    );
}
```

The important concept is:

```
Keycloak  
  
ADMIN  
  ↓  
ROLE_ADMIN  
  
USER  
  ↓  
ROLE_USER
```

52. Retiring JwtAuthFilter

At this point, our old:

```
JwtAuthFilter
```

is no longer needed.

Neither is:

```
JwtUtil
```

assuming they were created solely for the previous JWT implementation.

The cleanup should include:

```
Delete JwtAuthFilter.java  
Delete JwtUtil.java  
Delete JJWT dependencies  
Delete jwt.secret
```

Do not leave two competing JWT validation mechanisms in the same Gateway without a specific migration reason.

53. Why Not Keep Both?

A common beginner reaction is:

Why not keep the old filter just in case?

Because now we could accidentally have:

```
Old authentication
+
New authentication
```

This increases complexity.

It may also create inconsistent security behavior.

For our controlled training platform:

```
Clean cutover
```

is appropriate.

In a real production migration with existing clients and tokens, a temporary parallel migration strategy may be appropriate.

54. Preserving User Context Headers

Earlier we added:

```
X-User-Id
X-User-Role
```

to requests sent to downstream services.

We can preserve that architectural pattern.

But the source of the information changes.

Before:

```
JwtAuthFilter
  ↓
manually parsed JWT
  ↓
X-User-Id
```

Now:

```
Spring Security
  ↓
validated Jwt
  ↓
X-User-Id
```

55. User Context Enrichment

A simplified Gateway filter can use the authenticated JWT:

```
@Component
public class UserContextEnrichmentFilter
    implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(
        ServerWebExchange exchange,
        GatewayFilterChain chain) {

        return ReactiveSecurityContextHolder
            .getContext()

            .map(context ->
                context.getAuthentication()
                    .getPrincipal())

            .cast(Jwt.class)

            .map(jwt ->
                exchange.mutate()
                    .request(
                        exchange.getRequest()
                            .mutate()
                            .header(
```

```

                                "X-User-Id",
                                jwt.getSubject()
                                )
                                .build()
                                )
                                .build()
                                )
                                .defaultIfEmpty(exchange)
                                .flatMap(chain::filter);
    }

    @Override
    public int getOrder() {
        return Ordered.LOWEST_PRECEDENCE - 1;
    }
}

```

The important concept is not the exact filter code.

The important concept is:

```

Spring Security validates identity
      ↓
Gateway can derive trusted user context
      ↓
Downstream request

```

56. Important Security Warning About Headers

If downstream services trust:

```

X-User-Id
X-User-Role

```

then clients must not be able to inject their own values.

For example, a malicious client should not be able to send:

```
X-User-Role: ADMIN
```

and have the Gateway simply forward it.

The Gateway should:

1. Remove untrusted incoming identity headers.
2. Authenticate the token.
3. Generate the trusted headers itself.
4. Forward them downstream.

Even better, downstream services should also have their own security boundary where appropriate.

57. Service-to-Service Authentication

Now return to our earlier problem:

```
Order Service
      ↓
Inventory Service
```

There is no customer.

Therefore:

```
Order Service
```

needs its own identity.

58. Order Service Token Client

The simple conceptual algorithm is:

```
getToken()

IF cached token is still valid
    return cached token

ELSE
```

```
request new token from Keycloak
cache it
return token
```

59. Token Caching

Never request a new Client Credentials token for every API call if you can avoid it.

Bad:

```
API call
↓
Get token
↓
API call

API call
↓
Get token
↓
API call
```

Better:

```
Get token
↓
Cache token
↓
API call
↓
API call
↓
API call
↓
Refresh shortly before expiration
```

60. Refreshing Before Expiration

Suppose:

```
expires_in = 300 seconds
```

We should not necessarily wait until the exact expiration second.

A simple strategy:

```
refresh 30 seconds before expiry
```

Conceptually:

```
expiresAt =  
    Instant.now()  
        .plusSeconds(expiresIn - 30);
```

In production, use appropriate synchronization to avoid a "token refresh stampede" when many threads request a token simultaneously.

61. Using the Token

The service then calls:

```
GET /api/inventory  
Authorization: Bearer eyJ...
```

Inventory Service receives:

```
Bearer token
```

Spring Security validates it.

62. Customer Token vs Service Token

This is one of the most important concepts in the entire session.

Customer request

```
graph TD; Customer --> Keycloak; Keycloak --> Token[Customer Access Token]; Token --> Gateway; Gateway --> OrderService[Order Service];
```

The token represents:

```
customer1
```

Scheduled service request

```
graph TD; Scheduler --> OrderService[Order Service]; OrderService --> Keycloak; Keycloak --> Token[Service Access Token]; Token --> InventoryService[Inventory Service];
```

The token represents:

```
order-service
```

There is no customer.

63. Feign and Service Tokens

Earlier we had:

```
FeignJwtInterceptor
```

that propagated the incoming customer's JWT.

That is correct when there is a user request:

```
Customer
  ↓
Gateway
  ↓
Order
  ↓
Inventory
```

But consider:

```
Scheduler
  ↓
Order
  ↓
Inventory
```

There is no incoming user JWT.

Therefore:

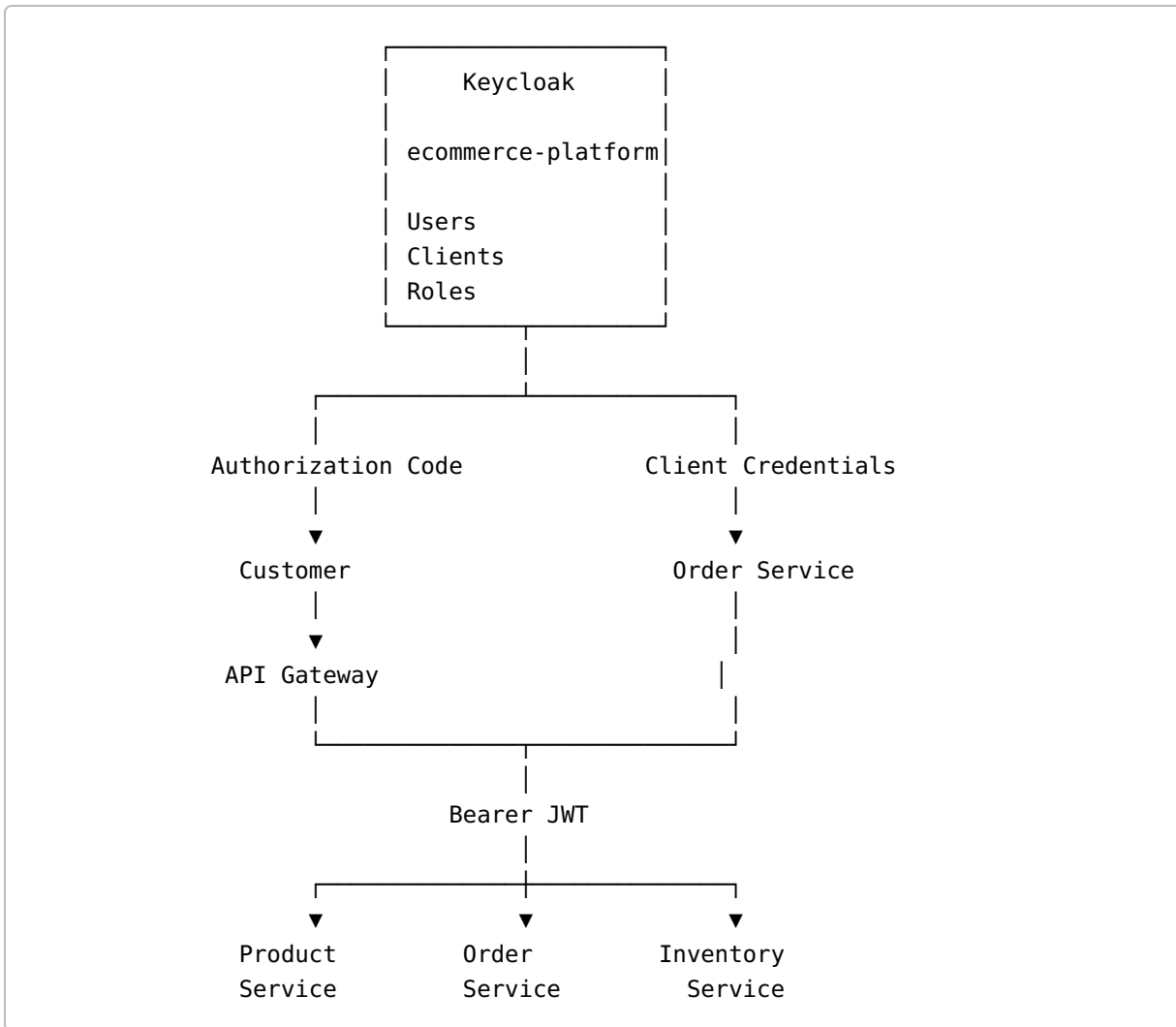
```
If user token exists:
    propagate appropriate user context/token

Otherwise:
    authenticate Order Service itself
    using Client Credentials
```

This distinction is essential.

64. Complete Security Architecture

After Sessions 19–20, our architecture looks like:



65. End-to-End Example 1 — Customer

Customer wants to view products.

1. Customer opens application
2. Application redirects to Keycloak
3. Keycloak authenticates customer
4. Keycloak returns authorization code
5. Client exchanges code for tokens

6. Client obtains access token
7. Request reaches Gateway
8. Gateway validates JWT
9. Gateway routes request to Product Service
10. Product Service returns products

66. End-to-End Example 2 — Admin

Admin requests:

```
GET /api/admin/products
```

The token contains:

```
ADMIN
```

Spring Security maps:

```
ADMIN  
↓  
ROLE_ADMIN
```

The Gateway checks:

```
.hasRole("ADMIN")
```

Result:

```
ROLE_ADMIN  
↓  
Allowed
```

67. End-to-End Example 3 — Normal User

A normal user requests:

```
GET /api/admin/products
```

Token:

```
USER
```

Spring Security maps:

```
ROLE_USER
```

The route requires:

```
ROLE_ADMIN
```

Therefore:

```
403 Forbidden
```

The user is authenticated.

But they are not authorized.

This demonstrates the difference:

```
401 Unauthorized
=
Authentication problem

403 Forbidden
=
Authenticated but not authorized
```

68. End-to-End Example 4 — Service-to-Service

A scheduled job runs:

```
Order Service
  |
  | no user
  ▼
Keycloak
  |
  | client_credentials
  ▼
Access Token
  |
  ▼
Inventory Service
```

Inventory validates:

```
signature
issuer
expiration
```

and applies any required authorization rules.

69. Common Mistakes

Mistake 1

Using:

```
KEYCLOAK_ADMIN
```

with a current Keycloak image without checking the image/version documentation.

Lesson

Keycloak container configuration is version-sensitive.

Mistake 2

Using:

```
localhost
```

inside Docker.

Remember:

```
localhost
```

means:

this container/machine

If Gateway is running in another container:

```
localhost:8180
```

does not mean Keycloak.

Use the Compose service name:

```
keycloak:8080
```

when the Gateway is also inside the Compose network.

70. Common Mistake — Issuer URI and Docker

Suppose Keycloak issues:

```
iss=http://localhost:8180/realms/ecommerce-platform
```

but Gateway runs inside Docker.

Gateway trying:

```
http://localhost:8180
```

will try to connect to itself.

This can fail.

Therefore, when containerizing the entire system, carefully design:

```
External Keycloak URL  
Internal Keycloak URL  
Issuer configuration  
DNS/service names
```

This becomes especially important in production deployments.

71. Common Mistake — Trusting the ID Token as an API Token

Do not automatically use:

```
id_token
```

as:

```
Authorization: Bearer ...
```

The access token is intended for API authorization.

The ID token is primarily an OpenID Connect identity artifact for the client.

72. Common Mistake — Keeping the Old JwtAuthFilter

Once Resource Server is configured, students may leave:

```
JwtAuthFilter
```

in the project.

That is not necessary for this course architecture.

The goal is:

```
Session 3:  
hand-written JWT validation
```

↓

```
Sessions 19–20:  
Keycloak + Spring Security
```

73. Common Mistake — Sharing the Client Secret

Never place:

```
client_secret
```

in:

```
JavaScript  
HTML  
mobile application source  
Git repository  
public configuration
```

A confidential client secret belongs in secure server-side configuration.

74. Common Mistake — Hardcoding Secrets

This is acceptable for a classroom demonstration:

```
client-secret: demo-secret
```

It is not acceptable as a production secret-management strategy.

Production systems should use mechanisms such as:

```
Environment variables  
Docker secrets  
Kubernetes Secrets  
Vault  
Cloud secret managers
```

75. Common Mistake — Thinking JWT Means Secure

JWT is a token format.

It does not automatically make an application secure.

Security depends on:

```
Who issued the token?  
Was the signature verified?  
Is the issuer trusted?  
Is the token expired?  
Is the token intended for this API?  
Is the user authorized?  
Are secrets protected?  
Are keys rotated?
```

76. Testing Checklist

After completing Session 20, verify the following.

Keycloak

```
[ ] Keycloak starts
[ ] Admin Console opens
[ ] ecommerce-platform realm exists
[ ] gateway client exists
[ ] customer1 exists
[ ] order-service client exists
```

Authorization Code

```
[ ] Login redirects to Keycloak
[ ] Customer can authenticate
[ ] Authorization code is returned
[ ] Code can be exchanged for tokens
[ ] Access token can be inspected
```

Resource Server

```
[ ] Resource Server dependency exists
[ ] issuer-uri configured
[ ] JwtAuthFilter removed
[ ] JwtUtil removed
[ ] old JJWT dependencies removed
[ ] jwt.secret removed
```

Authorization

```
[ ] Public product endpoint works
[ ] Authenticated endpoint rejects missing token
[ ] USER cannot access ADMIN endpoint
[ ] ADMIN can access ADMIN endpoint
```

Client Credentials

```
[ ] order-service client exists
[ ] service account enabled
[ ] client credentials token can be obtained
[ ] token represents order-service
[ ] service can call Inventory
```

77. Expected HTTP Results

Students should learn to recognize these results.

No authentication

401 Unauthorized

Meaning:

You have not successfully authenticated.

Authenticated but insufficient role

403 Forbidden

Meaning:

We know who you are,
but you are not allowed to perform this operation.

Valid authentication and authorization

200 OK

assuming the endpoint itself succeeds.

78. Practical Lab

Lab 1 — Start Keycloak

Run:

```
docker compose up -d keycloak
```

Verify:

```
http://localhost:8180
```

Lab 2 — Create Realm

Create:

```
ecommerce-platform
```

Lab 3 — Create Gateway Client

Create:

```
gateway
```

Configure it for the authorization-code flow.

Record:

```
Client ID  
Client Secret  
Redirect URI
```

Do not commit the secret to Git.

Lab 4 — Create Customer

Create:

```
customer1
```

Assign the appropriate user role.

Lab 5 — Perform Authorization Code Flow

Use the browser to:

```
Login  
↓  
Keycloak  
↓  
Authorization Code
```

Exchange the code for tokens.

Inspect the JWT.

Identify:

```
iss  
sub  
exp  
realm_access
```

Lab 6 — Configure Resource Server

Add:

```
spring-boot-starter-oauth2-resource-server
```

Configure:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8180/realms/ecommerce-platform
```

Lab 7 — Remove Old Security

Delete:

```
JwtAuthFilter
JwtUtil
JJWT dependencies
jwt.secret
```

Lab 8 — Add Role-Based Authorization

Configure:

```
/api/products
```

as public.

Configure:

```
/api/admin/**
```

as:

```
ADMIN only
```

Lab 9 — Test USER

Login as:

```
customer1
```

Try:

```
/api/admin/...
```

Expected:

```
403 Forbidden
```

Lab 10 — Test ADMIN

Create an administrator user.

Assign:

```
ADMIN
```

Login.

Try the same endpoint.

Expected:

```
200 OK
```

assuming the endpoint itself is implemented.

Lab 11 — Client Credentials

Create:

```
order-service
```

Configure its service account.

Request:

```
grant_type=client_credentials
```

Inspect the resulting token.

Confirm that it represents:

```
order-service
```

not:

```
customer1
```

Lab 12 — Service-to-Service Call

Implement:

```
Order Service
  ↓
Inventory Service
```

using:

```
Authorization: Bearer <service-token>
```

Verify that Inventory accepts the token.

79. Knowledge Check

Answer these questions without looking at the notes.

Question 1

Why was the Session 3 JWT implementation insufficient for a production identity system?

Question 2

What is the responsibility of Keycloak?

Question 3

What is the difference between an Authorization Server and a Resource Server?

Question 4

Why does the Authorization Code Flow use browser redirects?

Question 5

Why should a browser application not contain a client secret?

Question 6

What problem does PKCE solve?

Question 7

What is the difference between an Access Token and an ID Token?

Question 8

What does `iss` mean?

Question 9

What does `sub` mean?

Question 10

What is JWKS?

Question 11

Why is asymmetric signing better operationally than distributing the same HMAC secret to every service?

Question 12

What is the difference between:

Authentication

and:

Authorization

Question 13

Why does Client Credentials Flow not require a browser?

Question 14

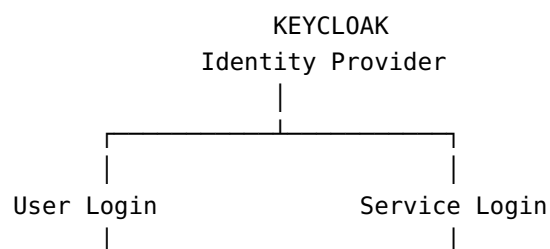
Who does a Client Credentials access token represent?

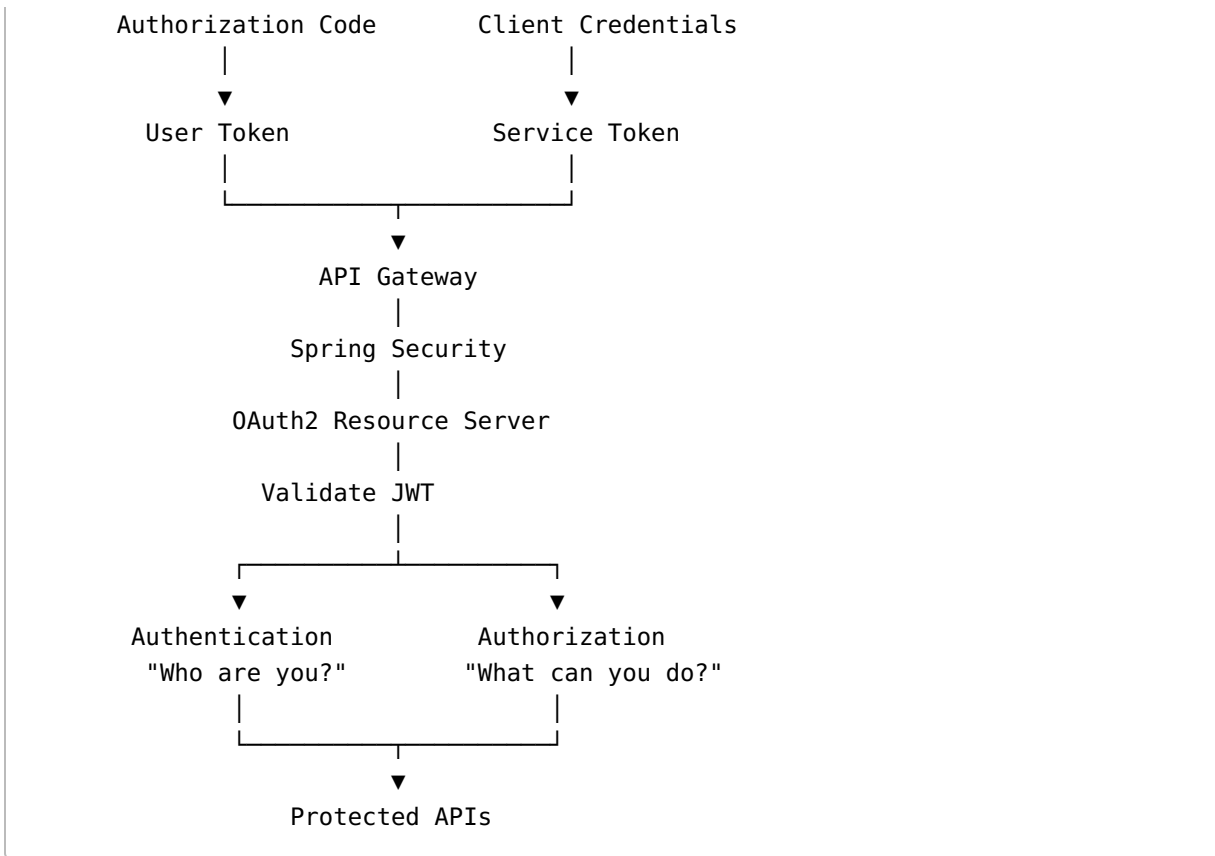
Question 15

Why should we remove `JwtAuthFilter` after migrating to Spring Security Resource Server?

80. Final Architecture to Remember

The most important concepts from these two sessions can be summarized as:





81. The Security Evolution of Our Course

The course security story is now:

Session 3

Hand-Rolled JWT

Shared Secret

+

JwtAuthFilter

+

Manual Validation

Sessions 19–20

Enterprise Identity

Keycloak

```
    +
OAuth2 / OIDC
    +
Authorization Code
    +
Client Credentials
    +
JWKS
    +
Spring Security Resource Server
    +
Role-Based Authorization
```

The important lesson is not simply:

"How do I configure Keycloak?"

The important lesson is:

Security responsibilities should be separated between the Identity Provider, OAuth2 Client, Resource Server, and authorization rules.

82. What You Should Be Able to Explain After These Sessions

If you understand Sessions 19–20, you should be able to explain this entire sentence:

"Keycloak acts as our Identity Provider and Authorization Server. Users authenticate through OpenID Connect using the Authorization Code Flow, while services authenticate themselves using Client Credentials. Our Gateway acts as an OAuth2 Resource Server and uses Spring Security to validate JWT access tokens using Keycloak's published public keys. Authorization is then performed by mapping Keycloak roles into Spring Security authorities."

If you can explain that sentence in your own words, you understand the core security architecture introduced in these sessions.

83. Professional Engineering Rules

Remember these rules when moving beyond the classroom:

Rule 1

Never hardcode production secrets.

Rule 2

Never put confidential client secrets in browser applications.

Rule 3

Do not implement your own JWT validation when a mature security framework can do it correctly.

Rule 4

Validate:

```
signature
issuer
expiration
```

and, where appropriate:

```
audience
```

Rule 5

Authentication and authorization are different problems.

Rule 6

A user's token and a service's token represent different security principals.

Rule 7

Do not trust identity headers supplied by clients.

Rule 8

When running distributed applications in Docker, understand the difference between:

```
localhost
```

and:

service-name

Rule 9

Keep Keycloak configuration version-pinned and reproducible.

Rule 10

For production, use secure secret storage, HTTPS, appropriate token lifetimes, key rotation, logging, monitoring, and a carefully designed logout/revocation strategy.

84. Session Completion Checklist

You have successfully completed Sessions 19–20 when you can demonstrate:

- [✓] Keycloak running in Docker
- [✓] ecommerce-platform realm created
- [✓] Gateway client created
- [✓] Customer created
- [✓] Authorization Code Flow understood
- [✓] Access Token inspected
- [✓] ID Token understood
- [✓] Client Credentials Flow understood
- [✓] order-service client created
- [✓] Service token obtained
- [✓] JWKS understood
- [✓] Spring Resource Server configured
- [✓] issuer-uri configured
- [✓] JwtAuthFilter removed

```
[✓] JwtUtil removed  
  
[✓] JJWT dependencies removed  
  
[✓] Keycloak roles mapped  
  
[✓] USER route tested  
  
[✓] ADMIN route tested  
  
[✓] 401 vs 403 understood  
  
[✓] Service-to-service authentication tested
```

85. Final Takeaway

The goal of these sessions is not to memorize Keycloak screens or copy Spring Security configuration.

The goal is to understand the architecture:

```
Who authenticates the user?  
↓  
Keycloak  
  
Who issues tokens?  
↓  
Keycloak  
  
Who validates access tokens?  
↓  
Resource Server / Gateway  
  
How does validation happen?  
↓  
JWT + public keys/JWKS  
  
Who decides what the user can do?  
↓  
Authorization rules + roles  
  
What happens when there is no user?  
↓  
Client Credentials
```

Who does the service token represent?

↓

The service itself

Once this model is clear, Keycloak and Spring Security become implementation tools for an architecture you already understand.

End of Sessions 19–20

Security — Keycloak, OAuth2, OIDC & Resource Server